#3 We are at the target node for the target key: we set current node to a key node to ensure it will store `s`.
#4 Since `s` is not the empty string, we can see if there is an edge matching it, even partially.
#5 Match case #4 (WRT figure 6.15). If there isn't any edge sharing a common prefix, not even the first character, with `s`, then we need to add a new edge, with label `s`, to a new key node.
#6 Match case #1. There is an edge whose label is a prefix to `s`: we just need to traverse the edge.
#7 Otherwise, we are in match case #2 or #3: there is a common prefix between the edge's label and `s`, but there are also characters in the edge's label not matching `s`.
Therefore, we need to break down the edge and create a split node.
#8 Update the outgoing edge for this node, with an edge pointing to the bridge node, and labeled with the common prefix.
#9 Add an edge from the bridge node to the former children of current node: the label will be the original edge's label, stripped of `commonPreifix`.
#10 Finally, we still need to recursively add the remaining part of the key to the bridge node. If `sSuffix` is the empty string, this corresponds to match case #3, otherwise to match case #2.

This node we add is called a *bridge node*, because it will be a bridge between the intermediate node corresponding to the common prefix of the two strings, and the paths leading to the final nodes for these strings. Bridge nodes are, obviously, bifurcation points, where the path from root branches out.